

Unit-3

- **SERVER-SIDE PROGRAMMING**

- Introduction to PHP - Basic Programming Concepts of PHP: Variables - Data-types - Constants -
- Scope of Variables - Type of Variables - Type Casting - Operators - Operators Precedence - References - Arrays - Control Structures: Branching - If statement - Switch statement - Looping: for Loop - while Loop - do while Loop - foreach Loop - Functions: User Defined Functions - Built-in Function - Functions for Variables - Script Controlling Functions - Array Functions - Date and Time Functions - Mathematical Functions - String Functions - PHP Server Variables; Working with form - Uploading files to Web Server using PHP.

What is PHP?

- PHP stands for “Hypertext Preprocessor” (earlier: Personal Home Page).
- It is a server-side scripting language used to create dynamic and interactive web pages.
- PHP scripts are executed on the server, and the result is sent to the browser as HTML.

Key Features

- Open-source and free
- Easy to learn and integrate with HTML
- Supports databases like MySQL, Oracle, PostgreSQL
- Platform-independent (works on Windows, Linux, macOS)
- Supports both procedural and object-oriented programming

Basic Syntax

- `<?php`
- `// PHP code goes here`
- `echo "Hello, World!";`
- `?>`
- PHP code is embedded within HTML files and executed on the server.

PHP Variables

- A variable is used to store data that can be used and changed throughout a program.
- All variables in PHP start with a dollar sign (\$).
- Variable names are case-sensitive.

- `<?php`
- `$name = "Diya";`
- `$age = 25;`
- `echo "My name is $name and I am $age years old.";`
- `?>`

- **Rules for Variable Names**

- Must start with \$ followed by a **letter or underscore**.
- Cannot start with a number.
- Can contain letters, numbers, and underscores (_).
- PHP variable names are **case-sensitive** (\$Name ≠ \$name).

- Variables in PHP
- Variables are containers for storing information (data). Think of them as labeled boxes that can hold different values.
- Key Rules & Characteristics:
- Starts with \$: All PHP variables must start with the dollar sign (\$). This is a fundamental rule.
- Case-Sensitive: Variable names are case-sensitive. \$name is different from \$Name.
- Naming Rules: After the \$ sign, a variable name must start with a letter (A-z) or an underscore (_). It cannot start with a number.
- No Spaces: Variable names cannot contain spaces.
- Dynamic Typing: PHP is a loosely typed language. This means you do not have to declare the variable's data type when you declare it. The data type is automatically determined by the value it holds.

- <?php
- \$userName = "Alex"; // A String variable
- \$age = 30; // An Integer variable
- \$_price = 9.99; // A Float variable (starts with underscore)
- \$isLoggedIn = true; // A Boolean variable
- echo "Welcome, " . \$userName . "! You are " . \$age . " years old.";
- // The '.' is the concatenation operator in PHP (used to join strings).
- ?>

PHP Data Types

- Scalar Types (Hold a single value):

Data Type	Description	Example
String	A sequence of characters. Can be enclosed in single (' ') or double (" ") quotes.	<code>\$name = "Jane Doe";</code>
Integer	A non-decimal number (whole number).	<code>\$count = 100;</code>
Float (or Double)	A number with a decimal point or in exponential form.	<code>\$pi = 3.14159;</code>
Boolean	Represents two possible states: TRUE or FALSE .	<code>\$isValid = true;</code>

- Compound Types (Hold multiple values):

Data Type	Description	Example
Array	Stores multiple values in a single variable. Values can be accessed via an index or key.	<code>\$colors = ["Red", "Green", "Blue"];</code>
Object	An instance of a user-defined class. Used for structured data and methods (functions).	<code>// Covered in Object-Oriented Programming (OOP)</code>

Data Type

NULL

Description

A variable with no value. It is the only possible value of the NULL type.

Example

```
$data = null;
```

Resource

Used to hold references to external resources, such as a database connection or a file handle.

```
// e.g., result of a database query
```

- Checking Data Type
- You can check the data type of a variable using the `var_dump()` function, which outputs structured information.
- PHP
- `<?php`
- `$value = 42;`
- `var_dump($value); // Output: int(42)`
- `?>`

PHP Data Types

- PHP automatically converts the variable type depending on the value assigned (**loosely typed language**).

Type	Description	Example
String	Sequence of characters	<code>\$name = "Deepa";</code>
Integer	Whole numbers	<code>\$age = 25;</code>
Float (Double)	Decimal numbers	<code>\$price = 99.99;</code>
Boolean	True/False values	<code>\$isAdmin = true;</code>
Array	Collection of values	<code>\$colors = array("Red", "Blue", "Green");</code>
Object	Instance of a class	<code>\$student = new Student();</code>
NULL	Represents a variable with no value	<code>\$value = NULL;</code>

- <?php
- \$name = "Alice"; // String
- \$age = 20; // Integer
- \$height = 5.4; // Float
- \$isStudent = true; // Boolean
- \$subjects = array("Math", "Science"); // Array
- \$value = NULL; // Null
- var_dump(\$subjects); // Displays type and value
- ?>

PHP Constants

- A **constant** is a name or identifier for a **fixed value** that cannot be changed during the script execution.
- **Declaring Constants**
- Use the **define()** function
- Constants **do not start with \$**
- Usually written in **uppercase letters**
- **Syntax:**
- `define(name, value, case_insensitive);`

- <?php
- define("SITE_NAME", "MyWebsite");
- define("PI", 3.14159);
- echo SITE_NAME; // Output: MyWebsite
- echo PI; // Output: 3.14159
- ?>

- **Features of Constants**

- Global by default (can be used anywhere in the script)
- Cannot be changed or undefined once set
- Using const Keyword (PHP 5.3+)
- <?php
- const GREETING = "Welcome to PHP!";
- echo GREETING;
- ?>

- <?php
- // Variables
- \$user = "John";
- \$age = 30;
- \$salary = 50000.50;
- \$isEmployed = true;
-
- // Constant
- define("COMPANY", "TechWorld");
-
- // Output
- echo "Name: \$user
";
- echo "Age: \$age
";
- echo "Salary: \$salary
";
- echo "Employed: " . (\$isEmployed ? "Yes" : "No") . "
";
- echo "Company: " . COMPANY;
- ?>

- **Line-by-line Explanation**

- **Line 1:**

- `<?php`
- This is the **PHP opening tag**.
- It tells the server that the following lines are written in PHP and must be interpreted by the PHP engine.
- Every PHP script begins with this tag.

- **Line 2:**

- `// Variables`
- This is a **comment**.
- Comments are ignored by PHP — they are used to describe or document code for humans.
- In this case, it indicates that the next few lines are **variable declarations**.

- **Line 3:**
- `$user = "John";`
- Declares a **variable** named `$user`.
- Assigns it the **string value** "John".
- Variables in PHP always start with the **dollar sign (\$)**.
- Now, `$user` holds the text "John".

- **Line 4:**
 - \$age = 30;
 - Declares a **variable \$age** a
- **Line 5:**
 - \$salary = 50000.50;
 - Declares a **variable \$salary** and assigns it a **floating-point (decimal)** number 50000.50.
 - Represents the user's salary.

- **Line 6:**
- `$isEmployed = true;`
- Declares a **Boolean variable** `$isEmployed` and assigns it the value `true`.
- Boolean values can only be `true` or `false`.
- Here it indicates that the user is employed.

- **Line 8:**
- `// Constant`
- Another **comment line** used for section labeling.
- Indicates that the next line defines a **constant**.

- **Line 9:**
- `define("COMPANY", "TechWorld");`
- Defines a **constant** named COMPANY.
- A **constant** stores a value that **cannot be changed** during program execution.
- The function `define(name, value)` is used for this purpose.
- Here, COMPANY is assigned the string "TechWorld".
- You do **not** use `$` with constants — they are referenced directly by name.

- **Line 11:**
- // Output
- Comment to indicate that the next section will **display output** using the echo statement.
- **Line 12:**
- echo "Name: \$user
";
- echo is used to **display text or variables** on the screen (browser).
- Inside the double quotes " ", PHP automatically replaces \$user with its value (John).
-
 is an **HTML line break tag**, used to move the next output to a new line in the browser.
- Output:
- Name: John

- **Line 13:**

- `echo "Age: $age <br>";`
- Prints the text "Age:" followed by the value of `$age` (30), then a line break.
- Output:
- Age: 30

- **Line 14:**
- `echo "Salary: $salary <br>";`
- Prints "Salary:" followed by the value of \$salary (50000.5).
- Output:
- Salary: 50000.5

- **Line 15:**
- `echo "Employed: " . ($isEmployed ? "Yes" : "No") . "<br>;`
- Let's break this down carefully 
- `"Employed: "` — A **string** to be displayed.
- `.` — The **concatenation operator**, used to join strings in PHP.
- `($isEmployed ? "Yes" : "No")` — A **ternary operator** that checks a condition.
 - If `$isEmployed` is true, it prints "Yes".
 - If `$isEmployed` is false, it prints "No".
- `<br>` — Adds a line break at the end.
- Since `$isEmployed` is true, the output will be:
- Employed: Yes

- **Line 16:**
- `echo "Company: " . COMPANY;`
- Prints "Company:" followed by the value of the constant `COMPANY`.
- `.` joins the string and the constant value.
- Output:
- Company: TechWorld

- **Line 17:**
- `?>`
- This is the **closing PHP tag**.
- It tells the PHP interpreter that the script ends here.
- In pure PHP files, it's optional, but commonly included for readability.

- **Final Output in Browser**

- Name: John
- Age: 30
- Salary: 50000.5
- Employed: Yes
- Company: TechWorld

Element	Type	Purpose	Example
\$user	Variable	Stores text	\$user = "John";
\$age	Variable	Stores integer	\$age = 30;
\$salary	Variable	Stores decimal	\$salary = 50000.50;
\$isEmployed	Variable	Stores boolean (true/false)	\$isEmployed = true;
COMPANY	Constant	Fixed value (cannot change)	define("COMPANY", "TechWorld");
echo	Keyword	Displays output	echo "Hello";
.	Operator	Concatenates strings	"Age: " . \$age
? :	Operator	Conditional expression	\$x > 10 ? "Yes" : "No"
 	HTML Tag	Break line	"Hello "

- Write a PHP program to print your name, department, and roll number.

- <?php
- \$name = "Ananya";
- \$department = "Computer Science";
- \$rollno = "23CS101";
- echo "Name: \$name
";
- echo "Department: \$department
";
- echo "Roll Number: \$rollno
";
- ?>

- What is the purpose of `<?php ... ?>` tags in PHP?

- The `<?php ... ?>` tags are used to tell the server where the PHP code starts and ends inside a web page.
- Any code written inside these tags is processed by the PHP interpreter on the server before the result is sent to the browser.

- How do you write comments in PHP?

Comments are lines in code that are **ignored by the PHP engine** — used only for explanation or notes.

Type	Syntax	Example
Single-line	<code>// comment</code>	<code>// This is a comment</code>
Single-line (alternate)	<code># comment</code>	<code># Another comment</code>
Multi-line	<code>/* ... */</code>	<code>/* This is a multi-line comment */</code>

- Write a PHP program to add two numbers and print the result.

- <?php
- \$num1 = 25;
- \$num2 = 15;
- \$sum = \$num1 + \$num2;
- echo "The sum of \$num1 and \$num2 is: \$sum";
- ?>

- What is the difference between echo and print in PHP?

Feature

Outputs

Return Value

Speed

Usage in Expressions

Example

echo

One or more strings

No

Slightly faster

 No

```
echo "Hi", " there";
```

print

Only one string

Returns 1

Slightly slower

 Yes

```
print "Hi";
```

- <?php
- \$name = "Alice";
- \$city = "Kochi";
- echo "Hello, \$name!
";
- print "Welcome to \$city.";
- ?>

Scope of Variables

- **Scope** means the part of the program where a variable can be **accessed or used**.
- **Global Scope:**
- Local
- Static

Scope	Definition	Access Rule	Example
Local	Declared inside a function.	Only accessible within that single function . Destroyed when the function finishes execution.	php <?php function test() { \$a = 5; // Local } ?>
Global	Declared outside any function.	Accessible everywhere except inside a function unless specifically imported.	php <?php \$b = 10; // Global function test() { // \$b cannot be used here } ?>
GLOBAL Keyword	Used inside a function to access a global variable.	Used to explicitly reference a global variable from within a local function scope.	php <?php \$c = 15; function useGlobal() { global \$c; echo \$c; // Outputs 15 } ?>
\$GLOBALS Array	A PHP built-in associative array containing references to all global variables.	Accessible from anywhere , including inside functions, without using the global keyword.	php <?php \$d = 20; function useGlobalsArray() { echo \$GLOBALS['d']; // Outputs 20 } ?>
Static	Declared inside a function using the static keyword.	The variable retains its last value between function calls, but is still only accessible locally.	php <?php function countCalls() { static \$count = 0; \$count++; return \$count; } // The variable is NOT reset ?>

- **Local:** Declared inside a function. Only accessible within that **single function**. Destroyed when the function finishes execution.
- `<?php function test()`
- `{ $a = 5; // Local }`
- `?>`

- **Global**
- Declared outside any function.
- Accessible everywhere **except inside a function** unless specifically imported.

- `<?php $b = 10; // Global function test()`
- `{ // $b cannot be used here }`
- `?>`

- **Static**

- Declared inside a function using the static keyword. The variable **retains its last value** between function calls, but is still only accessible locally.
- <?php
- function countCalls()
- { static \$count = 0; \$count++; return \$count; } // The variable is NOT reset
- ?>

- <?php
- function greet() {
 - static \$count = 0; // Static variable
 - \$count++;
 - echo "Hello! This is greeting number \$count
";
- }

- greet();
- greet();
- greet();
- ?>

Explanation Step by Step

- **function greet()**
- We define a function named **greet**.
- This function prints a message and counts how many times it has been called.

- **static \$count = 0;**
- \$count is declared as **static**.
- This means it will **remember its value** even after the function finishes.
- It is **initialized only once** — the first time the function is called.

- **\$count++;**
- Increases the value of \$count by 1 each time the function runs.

- **echo "Hello! This is greeting number \$count
";**
- Displays a message showing how many times the function has been called.

- Hello! This is greeting number 1
- Hello! This is greeting number 2
- Hello! This is greeting number 3
- **Why Static?**
- Without static, \$count would be set to **0** every time the function runs, so you'd always get "greeting number 1".
- With static, \$count **keeps increasing** each time.

Type	Description	Example
Local	Declared inside a function , used only inside that function	<pre>php\n<?php\nfunction test(){\n \$x = 10; // local variable\n echo \$x;\n}\ntest();\n// echo \$x; ❌ Error: undefined variable\n?></pre>
Global	Declared outside functions , accessible only if declared global inside the function	<pre>php\n<?php\n\$x = 20;\nfunction show(){\n global \$x;\n echo \$x;\n // use global keyword\n}\nshow();\n?></pre>
Static	Retains its value between function calls	<pre>php\n<?php\nfunction counter()\n{\n static \$count = 0;\n \$count++;\n echo \$count .\n \"
\";\n}\ncounter();\ncounter(); // retains value\n?></pre>

Type of Variables (Data Types)

- PHP supports several data types, but since it is a **loosely-typed language**, you don't declare the type; the interpreter determines it based on the value assigned.

Type	Description	Example
Scalar Types	Hold single values.	
Integer	Whole numbers (non-decimal).	\$age = 30;
Float	Numbers with a decimal point.	\$price = 19.99;
String	Sequence of characters.	\$name = "Alice";
Boolean	Represents truth values (true or false).	\$isLoggedIn = true;
Compound Types	Hold multiple values.	
Array	Stores multiple values under a single name (indexed or associative).	\$colors = ["Red", "Blue"];
Object	Instances of user-defined classes.	\$user = new User();
Special Types		
NULL	A variable with no value.	\$data = null;

Type	Example	Description
Integer	<code>\$x = 10;</code>	Whole number
Float / Double	<code>\$pi = 3.14;</code>	Decimal number
String	<code>\$name = "Deepa";</code>	Sequence of characters
Boolean	<code>\$isTrue = true;</code>	True or False
Array	<code>\$colors = array("red", "green", "blue");</code>	Collection of values
Object	<code>class Car { }</code>	Instance of a class
NULL	<code>\$value = null;</code>	Represents empty variable
Resource	Database or file reference	e.g., <code>\$file = fopen("a.txt", "r");</code>

Type Casting (Type Conversion)

- Changing one data type to another.
- <?php
- `$sum = "5" + 3; // PHP automatically treats "5" as an integer (Type Juggling)`
- `echo $sum; // Output: 8`
- ?>

- **Explicit Casting (Manual)** → done by the programmer

Type Casting	Example	Output
(int)	(int)"10.6"	10
(float)	(float)"10.6"	10.6
(string)	(string)123	"123"
(bool)	(bool)1	true

- <?php
- \$x = 10.6;
- \$y = (int)\$x; // Convert float → int
- echo \$y; // 10
- ?>

Operators in Programming

Operator	Description	Example	Result
+	Addition	<code>\$a + \$b</code>	Sum
-	Subtraction	<code>\$a - \$b</code>	Difference
*	Multiplication	<code>\$a * \$b</code>	Product
/	Division	<code>\$a / \$b</code>	Quotient
%	Modulus	<code>\$a % \$b</code>	Remainder
**	Exponentiation	<code>\$a ** \$b</code>	Power

- <?php
- \$a = 5; \$b = 2;
- echo \$a + \$b; // 7
- echo \$a ** \$b; // 25
- ?>

Assignment Operators

Operator

=

+=

-=

*=

/=

Example

`$x = 5;`

`$x += 3;`

`$x -= 2;`

`$x *= 4;`

`$x /= 2;`

Meaning

Assign value

`$x = $x + 3;`

`$x = $x - 2;`

`$x = $x * 4;`

`$x = $x / 2;`

Comparison Operators

Operator	Description	Example	Result
==	Equal	5 == "5"	true
===	Identical (value + type)	5 === "5"	false
!=	Not equal	5 != 3	true
<>	Not equal	5 <> 5	false
>	Greater than	7 > 3	true
<	Less than	3 < 7	true
>=	Greater or equal	7 >= 7	true

Logical Operators

Operator	Description	Example	Result
&&	AND	(\$x > 0 && \$y < 5)	true if both true
,		,	OR
!	NOT	!(\$x > 5)	reverses condition

String Operators

Operator	Description	Example	Result
.	Concatenation	\$a . \$b	Joins two strings
.=	Append	\$a .= \$b	Add string to existing one

- <?php
- \$a = "Hello ";
- \$b = "World!";
- echo \$a . \$b; // Hello World!
- ?>

Increment / Decrement Operators

Operator	Example	Meaning
<code>++\$X</code>	Pre-increment	Increments, then uses value
<code>\$X++</code>	Post-increment	Uses value, then increments
<code>--\$X</code>	Pre-decrement	Decrements, then uses value
<code>\$X--</code>	Post-decrement	Uses value, then decrements

Operator Precedence

- Defines **which operator executes first** when an expression has multiple operators. **Precedence (High → Low)**

Precedence	Operator	Example
1	()	(2 + 3) * 4 = 20
2	++, --	\$x++
3	**	\$a ** \$b
4	*, /, %	\$a * \$b + \$c
5	+, -, .	\$a + \$b * \$c
6	<, >, <=, >=, ==, ===	Comparison
7	&&, `	
8	=	Assignment

- **Precedence** dictates the order in which operators are evaluated in an expression. Higher precedence operators are executed first.
- **Parentheses ()**: Expressions inside parentheses are always evaluated first.
- **Multiplication/Division/Modulus (*, /, %)**
- **Addition/Subtraction (+, -)**
- **Comparison (<, >, <=, >=)**
- **Logical AND (&&)**
- **Logical OR (||)**
- **Assignment (=, +=, etc.)**

- <?php
- \$result = 5 + 3 * 2;
- // Multiplication (3 * 2 = 6) happens before Addition (5 + 6).
- echo \$result; // Output: 11
-
- \$result_P = (5 + 3) * 2;
- // Parentheses forces Addition (5 + 3 = 8) first.
- echo \$result_P; // Output: 16
- ?>

- <?php
- \$result = 10 + 5 * 2; // Multiplication first
- echo \$result; // Output: 20
- ?>

- **References in PHP**

- In PHP, a **reference** means two variables point to the **same memory location**.

- `<?php`

- `$a = 10;`

- `$b = &$a; // $b references $a`

- `$b = 20;`

- `echo $a; // 20 (changes reflected)`

- `?>`

- `$b` is not a copy — it's an alias for `$a`. Any change in `$b` affects `$a`.

Concept

Syntax Example

Result

Assignment by Value

```
$b = $a;
```

\$a and \$b are independent.
Changing \$a does **not** affect \$b.

Assignment by Reference

```
$b = &$a;
```

\$a and \$b are linked.
Changing \$a **will** change \$b,
and vice versa.

Example of Reference:

- `<?php`
- `$original = 50;`
- `$alias = &$original; // $alias is now a reference to $original`
- `$alias = 100; // Change $alias`
- `echo $original; // Output: 100 (The original variable also changed)`
- `?>`

Concept

Scope

Data Types

Type Casting

Operators

Precedence

References

Key Idea

Where variable can be used

Kind of data

Convert type

Perform operations

Order of evaluation

Two variables share same
memory

Example

global, local, static

int, float, string

(int)"10.5"

+, -, ., ==

* before +

\$b = &\$a;

Arrays and Control Structures (Branching & Looping)

- **What is an Array?**
- An **array** is a special variable that can **store multiple values** under a single name.
- `<?php`
- `$colors = array("Red", "Green", "Blue");`
- `echo $colors[0]; // Output: Red`
- `?>`

Types of Arrays

Type	Description	Example
Indexed Array	Uses numeric index (0, 1, 2, ...)	<pre>\$a = array("Apple", "Banana", "Mango");</pre>
Associative Array	Uses named keys instead of numbers	<pre>\$age = array("John"=>25, "Mary"=>30);</pre>
Multidimensional Array	Array containing one or more arrays	<pre>\$marks = array(array(80, 90, 70), array(85, 75, 65));</pre>

Indexed Array Example

- <?php
- \$fruits = array("Apple", "Mango", "Orange");
- echo \$fruits[1]; // Output: Mango

- // Loop through array
- foreach (\$fruits as \$f) {
- echo \$f . "
";
- }
- ?>

Associative Array Example

- `<?php`
- `$age = array("John"=>25, "Mary"=>30, "Alex"=>28);`
- `echo "Mary is " . $age["Mary"] . " years old.";`
- `?>`

Multidimensional Array Example

- <?php
- \$students = array(
 - array("John", 85, 90),
 - array("Mary", 75, 88),
 - array("Alex", 95, 92)
-);
- echo \$students[0][0] . " scored " . \$students[0][1];
- ?>

Array Functions

Function	Description	Example	Output
<code>count()</code>	Returns number of elements	<code>count(\$arr)</code>	Number
<code>sort()</code>	Sorts array ascending	<code>sort(\$arr)</code>	A-Z
<code>rsort()</code>	Sorts descending	<code>rsort(\$arr)</code>	Z-A
<code>array_push()</code>	Add item to end	<code>array_push(\$arr, "Grapes");</code>	Adds element
<code>array_pop()</code>	Removes last item	<code>array_pop(\$arr);</code>	Removes
<code>array_keys()</code>	Returns all keys	<code>array_keys(\$arr)</code>	Array of keys

- **count()** – **Count elements in an array**
- Returns the **number of elements** in an array.
- <?php
- \$fruits = array("Apple", "Banana", "Mango");
- echo count(\$fruits); // Output: 3
- ?>

- **array_push()** – **Add elements to the end**
- Appends one or more items to an array.
- <?php
- \$colors = array("Red", "Green");
- array_push(\$colors, "Blue", "Yellow");
- print_r(\$colors);
- ?>
- Output: Array ([0] => Red [1] => Green [2] => Blue [3] => Yellow)

- **array_pop()** – Remove last element
- Removes the **last item** from an array.
- <?php
- \$fruits = array("Apple", "Banana", "Cherry");
- array_pop(\$fruits);
- print_r(\$fruits);
- ?>
- Output: Array ([0] => Apple [1] => Banana)

- **array_shift()** – Remove first element
- Removes the **first element** from an array.
- <?php
- \$nums = array(10, 20, 30, 40);
- array_shift(\$nums);
- print_r(\$nums);
- ?>
- Output: Array ([0] => 20 [1] => 30 [2] => 40)

- **array_unshift()** – **Add elements to the beginning**
- Adds one or more elements to the **start** of an array.
- <?php
- \$nums = array(20, 30, 40);
- array_unshift(\$nums, 10);
- print_r(\$nums);
- ?>
- Output: Array ([0] => 10 [1] => 20 [2] => 30 [3] => 40)

- **sort()** – **Sort array (ascending)**
- Sorts values in ascending order (A–Z or smallest–largest).
- `<?php`
- `$names = array("Zara", "Alex", "John");`
- `sort($names);`
- `print_r($names);`
- `?>`
- Output: Array ([0] => Alex [1] => John [2] => Zara)

- **rsort()** – Sort array (descending)
- Sorts values in descending order (Z–A or largest–smallest).
- <?php
- \$marks = array(85, 90, 75);
- rsort(\$marks);
- print_r(\$marks);
- ?>
- Output: Array ([0] => 90 [1] => 85 [2] => 75)

- **asort()** – Sort associative array by value (ascending)
- Preserves key–value pairs while sorting **by value**.
- <?php
- \$age = array("John"=>25, "Mary"=>30, "Alex"=>22);
- asort(\$age);
- print_r(\$age);
- ?>
- Output:
Array ([Alex] => 22 [John] => 25 [Mary] => 30)

- **ksort()** – Sort associative array by key (ascending)
- Sorts **by keys** instead of values.
- <?php
- \$students = array("C"=>85, "A"=>90, "B"=>75);
- ksort(\$students);
- print_r(\$students);
- ?>
- Output:
Array ([A] => 90 [B] => 75 [C] => 85)

- **array_merge()** – **Combine two or more arrays**
- Merges arrays into a single array.
- <?php
- \$a1 = array("Red", "Green");
- \$a2 = array("Blue", "Yellow");
- \$result = array_merge(\$a1, \$a2);
- print_r(\$result);
- ?>
- Output:
Array ([0] => Red [1] => Green [2] => Blue [3] => Yellow)

- **array_combine()** – **Combine keys and values**
- Creates an array using one array for **keys** and another for **values**.
- <?php
- \$keys = array("name", "age", "city");
- \$values = array("John", 25, "Paris");
- \$person = array_combine(\$keys, \$values);
- print_r(\$person);
- ?>
- Output:
Array ([name] => John [age] => 25 [city] => Paris)

- **array_keys()** – **Get all keys from an array**
- Returns an array of all keys.
- <?php
- \$age = array("John"=>25, "Mary"=>30);
- \$keys = array_keys(\$age);
- print_r(\$keys);
- ?>
- Output: Array ([0] => John [1] => Mary)

- **array_keys()** – **Get all keys from an array**
- Returns an array of all keys.
- <?php
- \$age = array("John"=>25, "Mary"=>30);
- \$keys = array_keys(\$age);
- print_r(\$keys);
- ?>
- Output: Array ([0] => John [1] => Mary)

- **array_values()** – **Get all values from an array**
- Returns an array of all values.
- <?php
- \$age = array("John"=>25, "Mary"=>30);
- \$values = array_values(\$age);
- print_r(\$values);
- ?>
- Output: Array ([0] => 25 [1] => 30)

- **in_array()** – Check if a value exists
- Checks whether a value exists in an array.
- <?php
- \$colors = array("Red", "Blue", "Green");
- if(in_array("Blue", \$colors)){
- echo "Found Blue!";
- }
- ?>
- Output: Found Blue!

- **array_search()** – Search for a value and return its key
- Returns the key if the value is found.
- <?php
- \$age = array("John"=>25, "Mary"=>30, "Alex"=>28);
- echo array_search(30, \$age); // Output: Mary
- ?>

- **array_reverse()** – Reverse array order
- Reverses the elements of an array.
- <?php
- \$nums = array(1, 2, 3, 4, 5);
- \$rev = array_reverse(\$nums);
- print_r(\$rev);
- ?>
- Output: Array ([0] => 5 [1] => 4 [2] => 3 [3] => 2 [4] => 1)

- **array_slice()** – **Extract a part of an array**
- Returns a portion of an array.
- <?php
- \$fruits = array("Apple", "Banana", "Cherry", "Mango");
- \$part = array_slice(\$fruits, 1, 2);
- print_r(\$part);
- ?>
- Output: Array ([0] => Banana [1] => Cherry)

- **array_unique()** – Remove duplicate values
- Removes duplicate elements from an array.
- <?php
- \$nums = array(10, 20, 10, 30, 20);
- print_r(array_unique(\$nums));
- ?>
- Output: Array ([0] => 10 [1] => 20 [3] => 30)

- **array_sum()** – Calculate sum of array values
- Adds all numeric elements in an array.
- <?php
- \$numbers = array(10, 20, 30);
- echo array_sum(\$numbers); // Output: 60
- ?>

- **implode() and explode()**
- Convert between **string** ↔ **array**.
- **implode() – Convert array to string**
- <?php
- \$arr = array("Apple", "Banana", "Mango");
- \$str = implode(", ", \$arr);
- echo \$str; // Output: Apple, Banana, Mango
- ?>
- **explode() – Convert string to array**
- <?php
- \$str = "Red,Green,Blue";
- \$arr = explode(",", \$str);
- print_r(\$arr);
- ?>
- Output: Array ([0] => Red [1] => Green [2] => Blue)

CONTROL STRUCTURES

- Control structures **control the flow of execution** in a PHP program.
They are mainly of two types:
- **Branching (Decision Making)**
- **Looping (Repetition)**

BRANCHING STATEMENTS

- **1. if Statement**
- Executes a block of code if the condition is true.
- `<?php`
- `$age = 20;`
- `if ($age >= 18) {`
- `echo "You are eligible to vote.";`
- `}`
- `?>`

- **2. if...else Statement**

- Executes one block if condition is true, another if false.

- <?php

- \$num = 5;

- if (\$num % 2 == 0) {

- echo "Even number";

- } else {

- echo "Odd number";

- }

- ?>

- **if...elseif...else Statement**
- Used to test multiple conditions.
- <?php
- \$marks = 85;
- if (\$marks >= 90) {
- echo "Grade A";
- } elseif (\$marks >= 75) {
- echo "Grade B";
- } else {
- echo "Grade C";
- }
- ?>

- **switch Statement**

- Used when you have many possible conditions for one variable.

- <?php

- \$day = "Tue";

- switch(\$day) {

- case "Mon":

- echo "Monday";

- break;

- case "Tue":

- echo "Tuesday";

- break;

- case "Wed":

- echo "Wednesday";

- break;

- default:

- echo "Other day";

- }

- ?>

- **LOOPING STATEMENTS**

- Loops execute a block of code **repeatedly** until a condition is met.

- **1. while Loop**

- Repeats code while condition is true.

- <?php

- \$i = 1;

- while (\$i <= 5) {

- echo \$i . " ";

- \$i++;

- }

- ?>

- **2. do...while Loop**

- Executes code **at least once**, even if condition is false.
- <?php
- \$i = 1;
- do {
- echo \$i . " ";
- \$i++;
- } while (\$i <= 5);
- ?>

- **3. for Loop**

- Used when number of iterations is known.
- <?php
- for (\$i = 1; \$i <= 5; \$i++) {
- echo "Number: \$i
";
- }
- ?>

- **foreach Loop**

- Used for looping through **arrays**.

- <?php

- \$colors = array("Red", "Green", "Blue");

- foreach (\$colors as \$c) {

- echo \$c . "
";

- }

- ?>

- break and continue

Keyword	Description	Example
break	Stops the loop immediately	<code>if(\$i==3) break;</code>
continue	Skips current iteration and moves to next	<code>if(\$i==3) continue;</code>

- <?php
- for(\$i=1; \$i<=5; \$i++){
- if(\$i == 3) continue;
- echo \$i . " ";
- }
- ?>

Concept

Indexed Array

Associative Array

Multidimensional

if / else

switch

while / for / foreach

break / continue

Description

Numeric keys

Named keys

Arrays inside arrays

Conditional branching

Multiple conditions

Repetition

Control loop flow

Example

```
$arr = [1,2,3];
```

```
$age["John"]=25;
```

```
$a = [[1,2],[3,4]];
```

```
if($a>10)
```

```
switch($x)
```

```
for($i=0;$i<5;$i++)
```

Skips or stops loop

Functions in PHP

- Functions are **reusable code blocks** that perform a specific task.
- **User-Defined Functions:** You create these functions yourself.

- <?php
- function greet(\$name) {
- echo "Hello, \$name!
";
- }
- greet("John");
- greet("Mary");
- ?>

- Output:
- Hello, John!
- Hello, Mary!

- **function** greet(\$name)
- { **return** "Hello, " . \$name; }
- **echo** greet("Alice");
- Output???

- <?php
- function greetUser(\$name) { // \$name is the parameter
- echo "Hello, " . \$name . "!";
- }

- function addNumbers(\$num1, \$num2) {
- \$sum = \$num1 + \$num2;
- return \$sum; // Returns the result
- }

- greetUser("Alex"); // Function call
- \$total = addNumbers(10, 5); // \$total receives the returned value (15)
- ?>

Built-in Functions

- PHP has many **predefined functions**, such as:
- `strlen()` – Get string length
- `count()` – Count array elements
- `date()` – Display current date
- `sqrt()` – Square root of a number

Category	Description	Example Function	Purpose
Functions for Variables	Check variable status, type, and content.	<code>isset()</code>	Checks if a variable is set (declared and is not NULL).
		<code>empty()</code>	Checks if a variable is empty (0, false, "", null, or an empty array).
		<code>gettype()</code>	Returns the data type of a variable.
Script Controlling Functions	Affect script execution flow or state.	<code>die()</code> or <code>exit()</code>	Terminates the execution of the script.
		<code>header()</code>	Sends a raw HTTP header (often used for redirects).

Array Functions

Manipulate, search, or sort arrays.

`count()`

Returns the number of elements in an array.

`array_push()`

Adds one or more elements to the end of an array.

`in_array()`

Checks if a value exists in an array.

Date and Time Functions

Format, calculate, or display time data.

`date()`

Formats a local date and time.

`time()`

Returns the current Unix timestamp (seconds since 1/1/1970).

Mathematical Functions

Standard mathematical operations.

`round()`

Rounds a floating-point number.

`rand()`

Generates a random integer.

`sqrt()`

Returns the square root of a number.

String Functions

Manipulate, search, or format strings.

`strlen()`

Returns the length of a string.

`str_replace()`

Replaces all occurrences of a string within a string.

`trim()`

Removes whitespace or other predefined characters from both sides of a string.

- <?php
- echo strlen("PHP");
- ?>
- OUTPUT???

Functions for Variables

Function	Description	Example
<code>isset(\$var)</code>	Checks if a variable is set	<code>isset(\$x)</code>
<code>unset(\$var)</code>	Deletes a variable	<code>unset(\$x)</code>
<code>empty(\$var)</code>	Checks if a variable is empty	<code>empty(\$name)</code>
<code>gettype(\$var)</code>	Returns the data type	<code>gettype(\$x)</code>
<code>var_dump(\$var)</code>	Prints type and value (debugging)	<code>var_dump(\$x)</code>

- <?php
- \$age = 25;
- if(isset(\$age)) echo "Variable exists!
";
- echo gettype(\$age); // integer
- ?>

Script Controlling Functions

- These control the execution of PHP scripts.

Function	Description
<code>include 'file.php';</code>	Includes a file (continues if file missing)
<code>require 'file.php';</code>	Includes a file (stops if missing)
<code>exit("Message");</code>	Stops script execution
<code>die("Message");</code>	Same as <code>exit()</code>

Array Functions

Function	Description	Example
<code>count()</code>	Count elements	<code>count(\$arr)</code>
<code>sort()</code>	Sort ascending	<code>sort(\$arr)</code>
<code>rsort()</code>	Sort descending	<code>rsort(\$arr)</code>
<code>array_push()</code>	Add element	<code>array_push(\$arr, "item")</code>
<code>array_pop()</code>	Remove last element	<code>array_pop(\$arr)</code>
<code>array_merge()</code>	Combine arrays	<code>array_merge(\$a1, \$a2)</code>

- <?php
 - \$nums = array(10,20,30);
 - array_push(\$nums,40);
 - print_r(\$nums);
 - ?>
-
- Output: Array ([0] => 10 [1] => 20 [2] => 30 [3] => 40)

Line	Code / Concept	Explanation
1	<code><?php</code>	Starts the PHP script.
2	<code>\$nums = array(10, 20, 30);</code>	Creates an array named \$nums with three elements: 10, 20, and 30. So now \$nums = [10, 20, 30].
3	<code>array_push(\$nums, 40);</code>	The array_push() function adds one or more elements to the end of an array. Here, it adds 40 to \$nums. Now \$nums = [10, 20, 30, 40].
4	<code>print_r(\$nums);</code>	The print_r() function displays the array structure in a readable form. It prints all the array elements with their indexes.
5	<code>?></code>	Ends the PHP script.

- `array_push()` is used to **append values** at the end of an array.
- It can also add **multiple elements** at once:
- `array_push($nums, 50, 60);`
- `print_r()` is useful for **debugging arrays** and viewing their contents.

Date and Time Functions

Function	Description	Example	Output
<code>date("Y-m-d")</code>	Current date	<code>echo date("Y-m-d");</code>	2025-10-27
<code>date("h:i:s A")</code>	Current time	<code>echo date("h:i:s A");</code>	09:15:20 AM
<code>time()</code>	Current Unix timestamp	<code>echo time();</code>	1730000000
<code>date_default_timezone_set("Asia/Kolkata")</code>	Set timezone	<code>date_default_timezone_set("Asia/Kolkata");</code>	—

- <?php
- date_default_timezone_set("Asia/Kolkata");
- echo "Today: " . date("d-m-Y h:i:s A");
- ?>

- OUTPUT:

Mathematical Functions

Function	Description	Example	Output
<code>abs(-10)</code>	Absolute value	<code>echo abs(-10);</code>	10
<code>pow(2, 3)</code>	Power	<code>echo pow(2, 3);</code>	8
<code>sqrt(16)</code>	Square root	<code>echo sqrt(16);</code>	4
<code>round(3.6)</code>	Round to nearest integer	<code>echo round(3.6);</code>	4
<code>rand(1, 10)</code>	Random number between 1-10	<code>echo rand(1, 10);</code>	e.g. 7
<code>max(\$arr)</code>	Maximum value	<code>max(10, 20, 30)</code>	30
<code>min(\$arr)</code>	Minimum value	<code>min(10, 20, 30)</code>	10

String Functions

Function	Description	Example	Output
<code>strlen()</code>	String length	<code>strlen("PHP")</code>	3
<code>strtoupper()</code>	Convert to uppercase	<code>strtoupper("php")</code>	PHP
<code>strtolower()</code>	Convert to lowercase	<code>strtolower("HELLO")</code>	hello
<code>strrev()</code>	Reverse string	<code>strrev("hello")</code>	olleh
<code>strpos()</code>	Find position of substring	<code>strpos("hello", "e")</code>	1
<code>substr()</code>	Extract part of string	<code>substr("welcome", 0, 3)</code>	wel
<code>str_replace()</code>	Replace text	<code>str_replace("world", "PHP", "Hello world")</code>	Hello PHP
<code>trim()</code>	Remove spaces	<code>trim(" PHP ")</code>	PHP

- <?php
- \$name = " php programming ";
- echo strtoupper(trim(\$name));
- ?>

- Output: PHP PROGRAMMING

PHP Server Variables

- PHP provides a special superglobal array called **\$_SERVER** which contains information about headers, paths, and script locations.

- Common server variables:

- `$_SERVER['PHP_SELF']` : Path of the current script.
- `$_SERVER['SERVER_NAME']` : Name of server host (e.g., localhost or domain).
- `$_SERVER['HTTP_HOST']` : Contents of the Host header from the current request.
- `$_SERVER['REQUEST_METHOD']` : Request type: GET or POST.
- `$_SERVER['REMOTE_ADDR']` : Client/user's IP address.
- `$_SERVER['SCRIPT_FILENAME']` : Absolute path to the script.

Example

- `echo $_SERVER['PHP_SELF'];` // Output:
/folder/script.php
- `echo $_SERVER['SERVER_NAME'];` // Output: localhost
- `echo $_SERVER['REQUEST_METHOD'];` // Output: GET or POST
- Refer [w3schools.com](https://www.w3schools.com)

Example Program: Display Server Info

- `<?php`
- `echo "<h3>Server Information</h3>";`
- `echo "Current Script: " . $_SERVER['PHP_SELF'] . "<br>";`
- `echo "Server Name: " . $_SERVER['SERVER_NAME'] . "<br>";`
- `echo "Host Header: " . $_SERVER['HTTP_HOST'] . "<br>";`
- `echo "Request Method: " . $_SERVER['REQUEST_METHOD'] . "<br>";`
- `echo "Client IP: " . $_SERVER['REMOTE_ADDR'] . "<br>";`
- `?>`

- Current Script: /serverinfo.php
- Server Name: localhost
- Host Header: localhost
- Request Method: GET
- Client IP: 127.0.0.1

- `$_SERVER['PHP_SELF']` → Displays current file name.
- `$_SERVER['SERVER_NAME']` → Displays the server domain (like localhost).
- `$_SERVER['REQUEST_METHOD']` → Shows whether form is submitted using GET or POST.

- **\$_REQUEST**

- \$_REQUEST is a PHP super global variable which contains submitted form data, and all cookie data.
- \$_REQUEST is an array containing data from \$_GET, \$_POST, and \$_COOKIE.

Using \$_REQUEST on \$_POST Requests

- POST request are usually data submitted from an HTML form. When a user clicks the submit button, the form data is sent to a PHP file specified in the action attribute of the `<form>` tag.

```
<html>
<body>

<form method="post" action="demo_request.php">
  Name: <input type="text" name="fname">
  <input type="submit">
</form>

</body>
</html>
```

- When a user clicks the submit button, the form data is sent to a PHP file specified in the action attribute of the <form> tag. In the action file we can use the `$_REQUEST` variable to collect the value of the input field.
- `demo_request.php`
- `$name = $_REQUEST['fname'];`
- `echo $name;`

\$_POST

- \$_POST contains an array of variables received via the HTTP POST method.
- There are two main ways to send variables via the HTTP Post method:
 - HTML forms
 - JavaScript HTTP requests
- \$_POST in HTML Forms
 - A HTML form submits information via the HTTP POST method if the form's method attribute is set to "POST".

- `<html>`
- `<body>`
- `<form method="POST" action="demo_request.php">`
- `Name: <input type="text" name="fname">`
- `<input type="submit">`
- `</form>`
- `</body>`
- `</html>`

- When a user clicks the submit button, the form data is sent to a PHP file specified in the action attribute of the <form> tag.
- In the action file we can use the `$_POST` variable to collect the value of the input field.
- `demo_request.php`
- `$name = $_POST['fname'];`
- `echo $name;`

PHP \$_GET

- \$_GET contains an array of variables received via the HTTP GET method.
- There are two main ways to send variables via the HTTP GET method:
- Query strings in the URL
- HTML Forms

Working with Forms in PHP

- **1. Creating an HTML Form**

- A form collects user input and sends it to a PHP script using the **POST** or **GET** method.
- `<form action="welcome.php" method="post">`
- Name: `<input type="text" name="username"><br>`
- Email: `<input type="text" name="email"><br>`
- `<input type="submit" value="Submit">`
- `</form>`

- *Query string in the URL*
- A query string is data added at the end of a URL. In the link below, everything after the ? sign is part of the query string:
- `<a href="demo_phpfile.php?subject=PHP&web=W3schools.com">Test $GET</a>`
- The query string above contains two key/value pairs:
- subject=PHP
- web=W3schools.com
- In the PHP file we can use the `$_GET` variable to collect the value of the query string.

- The PHP file demo_phpfile.php:
- <html>
- <body>
- **<?php**
- echo "Study " . \$_GET['subject'] . " at " . \$_GET['web'];
- **?>**
- </body>
- </html>

- \$_GET in HTML Forms
- A HTML form submits information via the HTTP GET method if the form's method attribute is set to "GET"
- <html>
- <body>
- <form action="welcome_get.php" method="GET">
- Name: <input type="text" name="name">
- E-mail: <input type="text" name="email">
- <input type="submit">
- </form>
- </body>
- </html>

- When a user clicks the submit button, the form data is sent to a PHP file specified in the action attribute of the <form> tag.
- The form fields are sent to the PHP file, with your input, as query strings:
- `welcome_get.php?name=John&email=john@example.com`
- In the action file we can use the `$_GET` variable to collect the value of the input fields.

- PHP code inside the welcome_get.php page:
- <html>
- <body>
- Welcome <?php echo \$_GET["name"]; ?>

- Your email address is: <?php echo \$_GET["email"]; ?>
- </body>
- </html>

Uploading Files to Web Server

- **Steps in File Upload**
- Create an HTML form with attribute `enctype="multipart/form-data"`.
- Use `$_FILES` superglobal to access file data in PHP.
- Move uploaded file to a specific directory using `move_uploaded_file()`.

HTML Form for File Upload

- `<form action="upload.php" method="post" enctype="multipart/form-data">`
- Select file to upload:
- `<input type="file" name="myfile">`
- `<input type="submit" value="Upload File">`
- `</form>`

PHP Script: upload.php

- <?php
- if (isset(\$_FILES['myfile'])) {
- \$target_dir = "uploads/";
- \$target_file = \$target_dir . basename(\$_FILES["myfile"]["name"]);
-
- if (move_uploaded_file(\$_FILES["myfile"]["tmp_name"], \$target_file)) {
- echo "File ". htmlspecialchars(basename(\$_FILES["myfile"]["name"])) . "
has been uploaded.";
- } else {
- echo "Sorry, there was an error uploading your file.";
- }
- }
- ?>

\$_FILES Superglobal Explanation

Index

`$_FILES['myfile']['name']`

`$_FILES['myfile']['type']`

`$_FILES['myfile']['size']`

`$_FILES['myfile']['tmp_name']`

`$_FILES['myfile']['error']`

Description

Original filename on the client machine

MIME type (e.g., image/jpeg)

File size in bytes

Temporary filename stored on the server

Error code (0 means no error)

- Always validate file types (e.g., only allow .jpg, .png, .pdf).
Set a maximum file size limit.
Use htmlspecialchars() to prevent script injection.
Store uploaded files outside the web root if possible.

Concept	Description	Superglobal
Server Variables	Info about server & environment	\$_SERVER
Form Data	Collecting user input	\$_GET, \$_POST
File Uploads	Sending files to server	\$_FILES
Move Uploaded File	Move temp file to target folder	move_uploaded_file()

- PHP Global Variables - Superglobals
- [\\$GLOBALS](#)
- [\\$_SERVER](#)
- [\\$_REQUEST](#)
- [\\$_POST](#)
- [\\$_GET](#)

Working with Forms: Uploading Files to Web Server Using PHP

- To upload a file from a web form to a server, PHP uses the `$_FILES` superglobal array and a special HTML form encoding.
- 1. HTML Form for File Upload
- The form's method must be POST.
- The enctype must be multipart/form-data (required for file uploads).
- Use the `<input type="file">` field.

- `<form action="upload.php" method="post" enctype="multipart/form-data">`
 - `<label for="fileSelect">Select file:</label>`
 - `<input type="file" name="myfile" id="fileSelect">`
 - `<input type="submit" name="submit" value="Upload">`
 - `</form>`
-
- This form allows users to choose a file and upload it to the server.

- 2. PHP Script to Handle Upload (upload.php)
- Access the uploaded file using `$_FILES['myfile']`.
- Use `move_uploaded_file()` to copy the file from the temporary directory to your folder.
- Always validate the file type and size for security.


```
. if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
.     if (isset($_FILES['myfile']) && $_FILES['myfile']['error'] == 0) {  
.         $fileTmpPath = $_FILES['myfile']['tmp_name'];  
.         $fileName = $_FILES['myfile']['name'];  
.         $fileSize = $_FILES['myfile']['size'];  
.         $fileType = $_FILES['myfile']['type'];  
.         // Set where to store uploaded files  
.         $uploadDir = 'uploads/';  
.         $destPath = $uploadDir . $fileName;  
.         // Move the file to the uploads directory  
.         if (move_uploaded_file($fileTmpPath, $destPath)) {  
.             echo "File uploaded successfully!";  
.         } else {  
.             echo "File upload failed!";  
.         }  
.     }  
. }  
. }
```

- The `$_FILES` array holds details such as the original filename, file type, temporary file name, any errors, and file size. The server temporarily stores uploaded files until you move them to your desired location.
- `$_SERVER` gives you information about server and request.
- To upload files, you need a special form (`enctype="multipart/form-data"`) and process files with the `$_FILES` array in PHP.
- Always use security measures and validation for uploaded files to protect your server.

- Create a **Student Profile Form** with:
- Name, Email, Department fields
- File Upload for profile photo
- Display uploaded details on submission

HTML Form — student_form.html

- <!DOCTYPE html>
- <html lang="en">
- <head>
- <meta charset="UTF-8">
- <title>Student Profile Form</title>
- </head>
- <body>
- <h2>Student Profile Form</h2>
- <!-- Form to submit data and upload file -->
- <form action="profile_upload.php" method="post" enctype="multipart/form-data">
- <label>Name:</label>
- <input type="text" name="name" required>

- <label>Email:</label>
- <input type="email" name="email" required>

- <label>Department:</label>
- <select name="dept" required>
- <option value="">--Select Department--</option>
- <option value="Computer Science">Computer Science</option>
- <option value="Electronics">Electronics</option>
- <option value="Mechanical">Mechanical</option>
- <option value="Civil">Civil</option>
- </select>

- <label>Upload Profile Photo:</label>
- <input type="file" name="profilePhoto" accept="image/*" required>

- <input type="submit" value="Submit">
- </form>
- </body>
- </html>

PHP Script — profile_upload.php

```
• <?php
• // Folder to save uploaded photos
• $target_dir = "uploads/";
• if (!is_dir($target_dir)) {
•     mkdir($target_dir, 0777, true); // Create folder if not exists
• }
•
• $target_file = $target_dir . basename($_FILES["profilePhoto"]["name"]);
• $uploadOk = 1;
•
• // Check file type
• $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
•
• // Allow only image files
• $allowedTypes = array("jpg", "jpeg", "png", "gif");
• if (!in_array($imageFileType, $allowedTypes)) {
•     echo "Only JPG, JPEG, PNG & GIF files are allowed.<br>";
•     $uploadOk = 0;
• }
•
```

```

    // Check file size (limit: 2MB)
    if ($_FILES["profilePhoto"]["size"] > 2000000) {
        echo "Sorry, your file is too large.<br>";
        $uploadOk = 0;
    }

    // Move file if valid
    if ($uploadOk == 1) {
        if (move_uploaded_file($_FILES["profilePhoto"]["tmp_name"], $target_file)) {
            // Display student details
            echo "<h2>Student Profile Details</h2>";
            echo "<b>Name:</b> " . htmlspecialchars($_POST['name']) . "<br>";
            echo "<b>Email:</b> " . htmlspecialchars($_POST['email']) . "<br>";
            echo "<b>Department:</b> " . htmlspecialchars($_POST['dept']) . "<br><br>";
            echo "<b>Uploaded Profile Photo:</b><br>";
            echo "<img src='" . $target_file . "' width='150' height='150'>";
        } else {
            echo "Error uploading your file.";
        }
    } else {
        echo "File upload failed.";
    }
?>

```

Concept

enctype="multipart/form-data"

\$_POST

\$_FILES

move_uploaded_file()

htmlspecialchars()

Explanation

Required for uploading files through forms

Collects text input values (name, email, department)

Handles uploaded file details (name, type, size, temporary location)

Moves uploaded file from temporary folder to permanent folder

Prevents malicious code from being executed (security)

Output:

- Student Profile Details
- Name: Anjali Sharma
- Email: anjali@gmail.com
- Department: Computer Science
- [Profile photo displayed below]